

Практическое занятие 21: Создание HTML-шаблонов и динамическое содержимое

Цель занятия:

- Создать HTML-шаблоны для различных страниц.
 - Реализовать динамическое отображение данных, используя статические данные в представлениях.
-

Задачи занятия

1. Введение в шаблоны Django
 2. Создание базового шаблона `base.html`
 3. Создание наследуемых шаблонов
 4. Реализация динамического содержимого без моделей
 5. Создание дополнительных динамических элементов
 6. Запуск и проверка приложения
 7. Выполнение практического упражнения. Добавление нового шаблона с динамическими данными
-

Задача 1: Введение в шаблоны Django

1.1. Важность HTML-шаблонов в Django

- **Шаблоны** позволяют отделить логику приложения от представления данных пользователю.
- Даже без моделей можно создавать динамические веб-страницы, используя данные, переданные из представлений.
- Обеспечивают переиспользование кода через механизм наследования шаблонов.

1.2. Обзор системы шаблонов Django

- **Django Template Language (DTL)** — встроенный язык шаблонов Django.
- Позволяет использовать теги, фильтры и переменные для отображения динамического контента.
- Поддерживает наследование шаблонов, включение других шаблонов и использование контекстных процессоров.

Задача 2: Создание базового шаблона `base.html`

2.1. Создание директории для шаблонов

- Убедитесь, что у вас есть директория `templates/events/` в вашем приложении `events`.
- Если нет, создайте её:

```
your_project/
├── events/
│   ├── templates/
│   │   └── events/
│   │       └── (шаблоны будут здесь)
```

2.2. Создание файла `base.html`

- В директории `events/templates/events/` создайте файл `base.html`.

2.3. Наполнение файла `base.html`

```
<!-- events/templates/events/base.html -->

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Система управления мероприятиями{% endblock %}
</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'events/css/styles.css' %}">
</head>
<body>
    <nav>
        <a href="{% url 'events:home' %}">Главная</a> |
        <a href="{% url 'events:services' %}">Услуги</a> |
        <a href="{% url 'events:team' %}">Команда</a> |
        <a href="{% url 'events:gallery' %}">Галерея</a> |
        <a href="{% url 'events:contact_us' %}">Свяжитесь с нами</a>
    </nav>
    <div class="content">
        {% block content %}
        <!-- Основной контент будет вставлен здесь -->
        {% endblock %}
    </div>
```

```
</body>
</html>
```

Пояснение:

- `{% block title %}{% endblock %}`: Позволяет переопределять заголовок страницы в дочерних шаблонах.
- `{% block content %}{% endblock %}`: Место, где будет вставлен основной контент из дочерних шаблонов.
- `{% load static %}`: Загружает тег `static` для доступа к статическим файлам.
- **Навигация**: Использует теги `{% url %}` для создания ссылок на маршруты по их именам.

2.4. Обновление файла стилей `styles.css`

- Перейдите в директорию для статических файлов: `events/static/events/css/`.
- Обновите файл `styles.css` в этой директории.

Пример содержимого `styles.css`:

```
/* events/static/events/css/styles.css */

body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
}

nav {
    background-color: #333;
    padding: 10px;
}

nav a {
    color: #ffffff;
    margin-right: 15px;
    text-decoration: none;
}

nav a:hover {
    text-decoration: underline;
}

.content {
    padding: 20px;
}
```

```
h1, h2 {
    color: #333;
}

ul {
    list-style-type: none;
    padding: 0;
}

li {
    margin-bottom: 10px;
}
```

2.5. Настройка статических файлов в `settings.py`

- Откройте `your_project/settings.py` и убедитесь, что настройки для статических файлов указаны правильно:

```
# your_project/settings.py

STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'events/static']
```

Задача 3: Создание наследуемых шаблонов

3.1. Обновление шаблона `home.html`

Шаги:

- Откройте файл `events/templates/events/home.html`.
- Измените его содержимое следующим образом:

```
<!-- events/templates/events/home.html -->

{% extends 'events/base.html' %}

{% block title %}Домашняя страница{% endblock %}

{% block content %}
    <h1>Добро пожаловать на сайт мероприятий!</h1>
    <p>Здесь вы найдете информацию о наших последних событиях.</p>
{% endblock %}
```

3.2. Обновление других шаблонов

Повторите аналогичные шаги для следующих шаблонов:

3.2.1. services.html

```
<!-- events/templates/events/services.html -->

{% extends 'events/base.html' %}

{% block title %}Услуги{% endblock %}

{% block content %}
    <h1>Наши услуги</h1>
    <ul>
        {% for service in services %}
            <li>{{ service }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

3.2.2. team.html

```
<!-- events/templates/events/team.html -->

{% extends 'events/base.html' %}

{% block title %}Команда{% endblock %}

{% block content %}
    <h1>Наша команда</h1>
    <ul>
        {% for member in team %}
            <li>{{ member.name }} - {{ member.position }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

3.2.3. gallery.html

```
<!-- events/templates/events/gallery.html -->

{% extends 'events/base.html' %}

{% block title %}Галерея{% endblock %}

{% block content %}
    <h1>Галерея мероприятий</h1>
```

```
<p>Фотографии и видео с наших мероприятий.</p>
{% endblock %}
```

3.2.4. contact_us.html

```
<!-- events/templates/events/contact_us.html -->

{% extends 'events/base.html' %}

{% block title %}Свяжитесь с нами{% endblock %}

{% block content %}
    <h1>Свяжитесь с нами</h1>
    <p>Здесь вы можете разместить форму обратной связи или контактную
информацию.</p>
{% endblock %}
```

3.3. Сохраните изменения

- Убедитесь, что вы сохранили все изменения в шаблонах.

Задача 4: Реализация динамического содержимого без моделей

Поскольку модели еще не введены, мы будем использовать статические данные, передаваемые из представлений в шаблоны.

4.1. Обновление представлений в events/views.py

Шаги:

1. Откройте файл events/views.py.
2. Добавьте или обновите следующие представления:

```
# events/views.py

from django.shortcuts import render

def home(request):
    return render(request, 'events/home.html')

def services(request):
    services_list = [
        'Организация мероприятий',
```

```

        'Аренда оборудования',
        'Кейтеринг',
        'Развлекательные программы',
    ]
    return render(request, 'events/services.html', {'services':
services_list})

team_members = [
    {'name': 'Иван Иванов', 'position': 'Директор'},
    {'name': 'Петр Петров', 'position': 'Менеджер проектов'},
    {'name': 'Светлана Смирнова', 'position': 'Координатор
мероприятий'},
]

def team(request):
    return render(request, 'events/team.html', {'team': team_members})

def gallery(request):
    return render(request, 'events/gallery.html')

def contact_us(request):
    return render(request, 'events/contact_us.html')

```

4.2. Использование переданных данных в шаблонах

- В шаблонах `services.html` и `team.html` мы уже используем переменные `services` и `team`, переданные из представлений.
- Убедитесь, что имена переменных в представлениях и шаблонах совпадают.

Задача 5: Создание дополнительных динамических элементов

5.1. Добавление списка мероприятий на главную страницу

Поскольку мы пока что не используем модели, мы можем передать статический список мероприятий.

Шаги:

1. Обновите представление `home` в `events/views.py`:

```

events = [
    {'id': 1, 'title': 'Концерт классической музыки', 'date':
'12.01.2025 19:00', 'description': 'Концерт с участием известных
исполнителей.', 'location': 'Концертный зал', 'organizer': 'Иван Иванов'},

```

```

        {'id': 2, 'title': 'Выставка современного искусства', 'date':
'02.02.2025 10:00', 'description': 'Выставка работ современных художников.',
'location': 'Галерея искусств', 'organizer': 'Петр Петров'},
        {'id': 3, 'title': 'Театральная постановка', 'date': '08.03.2025
18:30', 'description': 'Постановка классического произведения.', 'location':
'Драматический театр', 'organizer': 'Светлана Смирнова'},
    ]

def home(request):
    return render(request, 'events/home.html', {'events': events})

```

2. Обновите шаблон `home.html` для отображения списка мероприятий:

```

{% extends 'events/base.html' %}

{% block title %}Домашняя страница{% endblock %}

{% block content %}
    <h1>Добро пожаловать на сайт мероприятий!</h1>
    <p>Здесь вы найдете информацию о наших последних событиях.</p>

    <h2>Предстоящие мероприятия</h2>
    <ul>
        {% for event in events %}
            <li>
                <strong>{{ event.title }}</strong> - {{ event.date }}
            </li>
        {% empty %}
            <li>Нет предстоящих мероприятий.</li>
        {% endfor %}
    </ul>
{% endblock %}

```

5.2. Создание страницы деталей мероприятия

Поскольку у нас нет моделей, мы можем создать представление, которое будет принимать `event_id` и возвращать соответствующее мероприятие из списка.

Шаги:

1. Добавьте представление `event_detail` в `events/views.py`:

```

def event_detail(request, event_id):
    event = next((item for item in events if item['id'] == event_id), None)
    if event:
        return render(request, 'events/event_detail.html', {'event': event})

```



```
else:
    return render(request, 'events/event_not_found.html')
```

2. Измените шаблон event_detail.html :

```
<!-- events/templates/events/event_detail.html -->

{% extends 'events/base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
    <h1>{{ event.title }}</h1>
    <p><strong>Дата и время:</strong> {{ event.date }}</p>
    <p><strong>Место проведения:</strong> {{ event.location }}</p>
    <p><strong>Описание:</strong> {{ event.description }}</p>
    <p><strong>Организатор:</strong> {{ event.organizer }}</p>
{% endblock %}
```

3. Создайте шаблон event_not_found.html :

```
<!-- events/templates/events/event_not_found.html -->

{% extends 'events/base.html' %}

{% block title %}Мероприятие не найдено{% endblock %}

{% block content %}
    <h1>Мероприятие не найдено</h1>
    <p>К сожалению, мероприятие с указанным ID не найдено.</p>
{% endblock %}
```

4. Добавьте маршрут в events/urls.py :

```
# events/urls.py

urlpatterns = [
    path('', views.home, name='home'),
    path('events/', views.event_list, name='event_list'),
    path('events/<int:event_id>/', views.event_detail, name='event_detail'),
    path('services/', views.services, name='services'),
    path('team/', views.team, name='team'),
    path('gallery/', views.GalleryView.as_view(), name='gallery'),
    path('contact_us/', views.contact_us, name='contact_us'),
]
```

5. Обновите ссылки на мероприятия в шаблоне `home.html` :

```
<ul>
  {% for event in events %}
    <li>
      <a href="{% url 'events:event_detail' event.id %}">{{
event.title }}</a> - {{ event.date }}
    </li>
  {% empty %}
    <li>Нет предстоящих мероприятий.</li>
  {% endfor %}
</ul>
```

Задача 6: Запуск и проверка приложения

6.1. Запуск сервера разработки

```
python manage.py runserver
```

6.2. Проверка работы приложения

- Перейдите на главную страницу:
 - <http://127.0.0.1:8000/>
- Убедитесь, что список мероприятий отображается корректно.
- Кликните на название мероприятия, чтобы перейти на страницу деталей мероприятия.
- Убедитесь, что информация о мероприятии отображается правильно.
- Попробуйте перейти по несуществующему `event_id`, например:
 - <http://127.0.0.1:8000/events/99/>
- Убедитесь, что отображается страница "Мероприятие не найдено".

Задача 7: Практическое упражнение

Задание для студентов:

- Создайте новый шаблон `about.html`, наследующийся от `base.html`.
- Отобразите в нём информацию о компании: историю, миссию, команду и т.д.

Шаги:

1. Создайте шаблон `about.html` в `events/templates/events/`:

```
<!-- events/templates/events/about.html -->

{% extends 'events/base.html' %}

{% block title %}О нас{% endblock %}

{% block content %}
    <h1>О нашей компании</h1>
    <p>Наша компания занимается организацией мероприятий с 2010 года.</p>
    <p>Наша миссия – создавать незабываемые события для наших клиентов.</p>
    <h2>Наша команда</h2>
    <ul>
        {% for member in team %}
            <li>{{ member.name }} – {{ member.position }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

2. Добавьте представление `about` в `events/views.py`:

```
def about(request):
    return render(request, 'events/about.html', {'team': team_members})
```

3. Добавьте маршрут в `events/urls.py`:

```
urlpatterns = [
    # ... другие маршруты ...
    path('about/', views.about, name='about'),
]
```

4. Добавьте ссылку в навигацию в `base.html`:

```
<nav>
    <!-- ... другие ссылки ... -->
    <a href="{% url 'events:about' %}">О нас</a>
</nav>
```

Заключение

Поздравляем! Вы успешно:

- Создали базовый шаблон `base.html` и настроили наследование шаблонов.
 - Обновили существующие шаблоны для использования `base.html`.
 - Реализовали динамическое отображение данных, используя статические данные в представлениях.
 - Создали дополнительные страницы и маршруты.
-

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- **Как механизм наследования шаблонов помогает в разработке?**
 - Позволяет переиспользовать общие элементы интерфейса, сокращает дублирование кода, облегчает внесение изменений.
 - **Как можно передавать данные из представления в шаблон без использования моделей?**
 - Используя статические данные или данные из других источников, передавая их через словарь контекста в функцию `render()`.
 - **Как можно дальше улучшить функциональность сайта без моделей?**
 - Добавить интерактивность с помощью JavaScript.
 - Использовать формы для сбора данных от пользователя и обработки их в представлениях.
 - Подключить внешние API для получения данных.
-

Полезные ресурсы

- [Документация Django: Шаблоны](#)
- [Django Template Language \(DTL\) Cheatsheet](#)
- [Основы представлений в Django](#)
- [Настройка URL маршрутов](#)